

# SIMATS ENGINEERING

## Department of Computer Science & Engineering

### ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

|                       |                                                                                                                                                                                                                                                 |                      |                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------|
| <b>Course Code:</b>   | CSA12                                                                                                                                                                                                                                           | <b>Course Title:</b> | Computer Architecture                                                 |
| <b>CO Assessed:</b>   | CO5                                                                                                                                                                                                                                             | <b>Assessment:</b>   | ASSESSMENT TOOL2-<br>PSEUDOCODE<br>WRITING TASK<br>(ALGORITHM DESIGN) |
| <b>Weightage:</b>     | 30%                                                                                                                                                                                                                                             | <b>Total Marks:</b>  | 25                                                                    |
| <b>CO5 Statement:</b> | <i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i> |                      |                                                                       |
| <b>Student Name:</b>  | G. Naveen Kumar Reddy                                                                                                                                                                                                                           | <b>Reg.No.:</b>      | 192525288                                                             |
| <b>Department:</b>    | AIML                                                                                                                                                                                                                                            | <b>Date:</b>         |                                                                       |

## ANNEXURE A

### Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

#### ***Code of Conduct:***

I **G. Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student: G. Naveen Kumar Reddy***

## RUBRICS FOR CALCULATION

| Criteria                         | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|----------------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| <b>Problem Understanding</b>     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| <b>Algorithm Design</b>          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| <b>Use of Control Structures</b> | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| <b>Pseudocode Structure</b>      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| <b>Completeness</b>              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

# Answers

## Question 1: DMA-based Data Transfer Algorithm

### Problem understanding:

Inputs are the source I/O device address, destination memory address, transfer length and device id; the output is a completion status and byte count. The constraint is that the CPU must only be involved at setup and at final interrupt notification — the actual word-by-word transfer runs inside the DMA controller hardware, not on the CPU.

### Pseudocode:

#### MODULE DMA\_Transfer

INPUT : source\_device\_address, destination\_memory\_address, transfer\_length, device\_id

OUTPUT: transfer\_status (SUCCESS / FAILURE), bytes\_transferred

CONSTRAINT: DMA channel must be idle before a new transfer is set up;  
CPU is involved only at setup and at completion (interrupt).

BEGIN

FUNCTION Setup\_DMA(source, destination, length, device\_id)

IF DMA\_Controller.status == BUSY THEN

RETURN ERROR "DMA channel busy"

END IF

DMA\_Controller.source\_register <- source

DMA\_Controller.destination\_register <- destination

DMA\_Controller.count\_register <- length

DMA\_Controller.mode <- BLOCK\_TRANSFER // or CYCLE\_STEALING

DMA\_Controller.device\_id <- device\_id

RETURN OK

END FUNCTION

FUNCTION Start\_DMA\_Transfer(source, destination, length, device\_id)

status <- Setup\_DMA(source, destination, length, device\_id)

IF status != OK THEN

RETURN status

END IF

Request\_Bus(device\_id) // ask bus arbiter for ownership

WAIT UNTIL Bus\_Granted(device\_id) == TRUE

DMA\_Controller.status <- BUSY

Enable\_DMA\_Interrupt(device\_id) // interrupt on completion / error

CPU.release\_control() // CPU is now free for other work

RETURN IN\_PROGRESS

END FUNCTION

// Runs autonomously inside the DMA controller hardware, NOT on the CPU

PROCEDURE DMA\_Hardware\_Loop()

WHILE DMA\_Controller.count\_register > 0 DO

IF Bus.available == TRUE THEN

byte <- Read(DMA\_Controller.source\_register)

Write(DMA\_Controller.destination\_register, byte)

DMA\_Controller.source\_register <- DMA\_Controller.source\_register + 1

DMA\_Controller.destination\_register <- DMA\_Controller.destination\_register + 1

DMA\_Controller.count\_register <- DMA\_Controller.count\_register - 1

```

ELSE
    WAIT_CYCLE()    // cycle stealing: yield the bus periodically to the CPU
END IF
END WHILE
Raise_Interrupt(DMA_COMPLETE, device_id)
END PROCEDURE

// Interrupt Service Routine - the only point where the CPU re-engages
ISR DMA_Complete_Handler(device_id)
    DISABLE_INTERRUPTS()
    IF DMA_Controller.error_flag == TRUE THEN
        Log_Error(device_id)
        transfer_status <- FAILURE
    ELSE
        transfer_status <- SUCCESS
        bytes_transferred <- original_length - DMA_Controller.count_register
    END IF
    DMA_Controller.status <- IDLE
    Release_Bus(device_id)
    Notify_Waiting_Process(device_id, transfer_status)
    ENABLE_INTERRUPTS()
    RETURN FROM INTERRUPT
END ISR
END MODULE

```

## Question 2: Vectored Interrupt Handling System

### Problem understanding:

Inputs are the interrupt request lines from N devices along with a priority table and a vector table; the output is that the correct interrupt service routine (ISR) is dispatched for the highest-priority pending device without the CPU having to poll each device in turn (vectoring provides the ISR address directly).

### Pseudocode:

#### MODULE Vectored\_Interrupt\_Handler

INPUT : interrupt\_request\_lines[N] from N devices, priority\_table[N], vector\_table[N]

OUTPUT: correct ISR dispatched and executed for the highest-priority pending request

#### DATA STRUCTURES

Priority\_Queue PQ      // ordered by device priority  
 Vector\_Table[device\_id]      // maps device\_id -> ISR entry address

#### BEGIN

PROCEDURE Interrupt\_Controller\_Loop()

    WHILE TRUE DO

        Pending <- Scan\_Interrupt\_Lines()      // active IRQ lines this cycle

        IF Pending is EMPTY THEN

            CONTINUE

        END IF

        FOR EACH device IN Pending DO

            PQ.insert(device, priority\_table[device])

        END FOR

```

        highest <- PQ.extract_max()          // vectored: top priority serviced first
        Service_Interrupt(highest)
    END WHILE
END PROCEDURE

FUNCTION Service_Interrupt(device_id)
    IF CPU.interrupt_enabled == FALSE THEN
        PQ.reinsert(device_id)              // defer until interrupts re-enabled
        RETURN
    END IF

    DISABLE_INTERRUPTS()                   // atomic vector fetch
    current_priority <- priority_table[device_id]
    Save_Context(CPU.registers, CPU.PC)

    vector_address <- Vector_Table[device_id] // direct fetch, no device polling
    ENABLE_HIGHER_PRIORITY_INTERRUPTS_ONLY(current_priority) // allow preemption

    JUMP_TO(vector_address)                // dispatch to device-specific ISR

    Restore_Context(CPU.registers, CPU.PC)
    Send_EOI(device_id)                    // end-of-interrupt to controller
    ENABLE_INTERRUPTS()
END FUNCTION

// Example device-specific service routines registered in Vector_Table
ISR Keyboard_ISR()
    data <- Read_Port(KEYBOARD_DATA)
    Buffer_Insert(keyboard_buffer, data)
    RETURN
END ISR

ISR Disk_ISR()
    IF Disk.error == TRUE THEN
        Handle_Disk_Error()
    ELSE
        Complete_Disk_Transfer()
    END IF
    RETURN
END ISR
END MODULE

```

### Question 3: Dynamic Memory-Mapped vs I/O-Mapped I/O Selection

#### Problem understanding:

Inputs are the device id together with its required latency and bandwidth, plus the currently measured bus latency/bandwidth; the output is the I/O mode chosen for that access. Large or high-bandwidth transfers favour memory-mapped I/O (unified addressing, DMA-friendly), while small, latency-critical single-word transfers favour isolated I/O-mapped access (dedicated IN/OUT instructions with less overhead).

#### Pseudocode:

##### MODULE IO\_Mode\_Selector

INPUT : device\_id, required\_latency, required\_bandwidth

OUTPUT: selected\_mode (MEMORY\_MAPPED / IO\_MAPPED) and the access performed

CONSTANTS: LATENCY\_LIMIT\_PMIO, LATENCY\_LIMIT\_MMIO, BANDWIDTH\_LIMIT\_PMIO

BEGIN

```
FUNCTION Measure_Current_Conditions(device_id)
  latency <- Get_Current_Bus_Latency(device_id)
  bandwidth <- Get_Current_Bus_Bandwidth(device_id)
  RETURN (latency, bandwidth)
END FUNCTION
```

```
FUNCTION Select_IO_Mode(device_id, req_latency, req_bandwidth)
  (latency, bandwidth) <- Measure_Current_Conditions(device_id)
```

```
  IF req_bandwidth > BANDWIDTH_LIMIT_PMIO THEN
    RETURN MEMORY_MAPPED // large/burst transfers: full address space + DMA support
```

```
  ELSE IF req_latency < LATENCY_LIMIT_PMIO AND req_bandwidth <= BANDWIDTH_LIMIT_PMIO
  THEN
```

```
    RETURN IO_MAPPED // small latency-critical word: dedicated IN/OUT, no MMU overhead
```

```
  ELSE IF latency > LATENCY_LIMIT_MMIO THEN
```

```
    RETURN IO_MAPPED // system bus congested -> fall back to isolated I/O bus
```

```
  ELSE
```

```
    RETURN MEMORY_MAPPED // default: unified addressing, simpler programming model
```

```
  END IF
```

```
END FUNCTION
```

```
PROCEDURE Perform_Access(device_id, address, data, direction, req_latency, req_bandwidth)
  mode <- Select_IO_Mode(device_id, req_latency, req_bandwidth)
```

```
  IF mode == MEMORY_MAPPED THEN
```

```
    IF direction == READ THEN data <- Load(address)
```

```
    ELSE Store(address, data)
```

```
  END IF
```

```
  ELSE // IO_MAPPED
```

```
    IF direction == READ THEN data <- IN(device_port(device_id))
```

```
    ELSE OUT(device_port(device_id), data)
```

```
  END IF
```

```
END IF
```

```
  Log_Mode_Decision(device_id, mode)
```

```
  RETURN data
```

```
END PROCEDURE
```

// Re-evaluate periodically as system conditions change

```
PROCEDURE Adaptive_Monitor()
```

```
  WHILE TRUE DO
```

```
    FOR EACH device IN Active_Devices DO
```

```
      new_mode <- Select_IO_Mode(device, device.req_latency, device.req_bandwidth)
```

```
      IF new_mode != device.current_mode THEN
```

```
        Switch_Mode(device, new_mode)
```

```
      END IF
```

```
    END FOR
```

```

        SLEEP(MONITOR_INTERVAL)
    END WHILE
END PROCEDURE
END MODULE

```

## Question 4: Bus Arbitration Algorithm (PCIe / USB / Thunderbolt)

### Problem understanding:

Inputs are the simultaneous bus requests from devices on three different interfaces, each with a priority and wait time; the output is a single bus grant per cycle. The algorithm combines priority ordering, round-robin fairness within equal priority, and aging (starvation prevention), then resolves across interfaces using PCIe > Thunderbolt > USB as the base precedence.

### Pseudocode:

#### MODULE Bus\_Arbitration

INPUT : request\_lines[N] from devices on PCIe / USB / Thunderbolt, priority[N], bus\_type[N]

OUTPUT: bus grant issued to exactly one device per arbitration cycle

#### DATA STRUCTURES

```

Request_Queue[bus_type]    // separate queue per interface
Round_Robin_Pointer[bus_type] // fairness pointer per interface

```

BEGIN

FUNCTION Arbitrate\_Bus()

WHILE TRUE DO

    requests <- Collect\_Requests() // {device\_id, bus\_type, priority, wait\_time}

    IF requests is EMPTY THEN

        Bus.owner <- NONE

        CONTINUE

    END IF

    pcie\_reqs <- FILTER(requests, bus\_type == PCIE)

    usb\_reqs <- FILTER(requests, bus\_type == USB)

    tbolt\_reqs <- FILTER(requests, bus\_type == THUNDERBOLT)

    pcie\_candidate <- Select\_Candidate(pcie\_reqs, PCIE)

    usb\_candidate <- Select\_Candidate(usb\_reqs, USB)

    tbolt\_candidate <- Select\_Candidate(tbolt\_reqs, THUNDERBOLT)

    // PCIe (system bus) > Thunderbolt (tunnels PCIe + DisplayPort) > USB

    winner <- Resolve\_Interface\_Priority(pcie\_candidate, tbolt\_candidate, usb\_candidate)

    IF winner != NULL THEN

        Grant\_Bus(winner.device\_id)

        Bus.owner <- winner.device\_id

        WAIT UNTIL Transfer\_Complete(winner.device\_id) OR Timeslice\_Expired()

        Revoke\_Bus(winner.device\_id)

    END IF

END WHILE

END FUNCTION

FUNCTION Select\_Candidate(request\_list, bus\_type)

    IF request\_list is EMPTY THEN

```

    RETURN NULL
END IF

highest_priority <- MAX(request_list.priority)
eligible <- FILTER(request_list, priority == highest_priority)

candidate <- Round_Robin_Select(eligible, Round_Robin_Pointer[bus_type])
Round_Robin_Pointer[bus_type] <- (candidate.index + 1) MOD LENGTH(eligible)

FOR EACH req IN request_list DO      // starvation prevention
    IF req != candidate THEN
        req.wait_time <- req.wait_time + 1
        IF req.wait_time > STARVATION_LIMIT THEN
            req.priority <- req.priority + 1    // dynamic priority boost (aging)
        END IF
    END IF
END FOR
RETURN candidate
END FUNCTION

FUNCTION Resolve_Interface_Priority(pcie_c, tbolt_c, usb_c)
    IF pcie_c != NULL THEN RETURN pcie_c
    ELSE IF tbolt_c != NULL THEN RETURN tbolt_c
    ELSE IF usb_c != NULL THEN RETURN usb_c
    ELSE RETURN NULL
    END IF
END FUNCTION
END MODULE

```

## Question 5: Hybrid I/O Communication Mechanism (Polling + DMA/Interrupt)

### Problem understanding:

Inputs are the device list with each device's data rate; the output is that every device is serviced by the mechanism suited to its speed class. Low-speed devices are handled by a lightweight polling loop (simple, low overhead for infrequent data), while high-speed devices are handed off to DMA with interrupt-driven completion notification, keeping the CPU free during the bulk of the transfer.

### Pseudocode:

#### MODULE Hybrid\_IO\_Manager

INPUT : device\_list[N], each with device.speed\_class and device.data\_rate

OUTPUT: every device serviced by the mechanism suited to its speed

CONSTANT: SPEED\_THRESHOLD // data-rate boundary between low- and high-speed handling

```

BEGIN
    PROCEDURE Classify_Device(device)
        IF device.data_rate < SPEED_THRESHOLD THEN
            device.mechanism <- POLLING
        ELSE
            device.mechanism <- DMA_INTERRUPT
        END IF
    END PROCEDURE

```



```

PROCEDURE Hybrid_Scheduler()
  FOR EACH device IN device_list DO
    Classify_Device(device)
  END FOR

  // High-speed devices: asynchronous DMA + interrupt path, minimal CPU load
  FOR EACH device IN device_list DO
    IF device.mechanism == DMA_INTERRUPT THEN
      Start_DMA_Transfer(device.source, device.destination, device.length, device.id)
    END IF
  END FOR

  // Low-speed devices: serviced through the polling loop below
  Polling_Loop()
END PROCEDURE

PROCEDURE Polling_Loop()
  WHILE System.running DO
    FOR EACH device IN device_list DO
      IF device.mechanism == POLLING AND Device_Ready(device) == TRUE THEN
        IF device.direction == READ THEN
          data <- Read_Device(device)
          Buffer_Store(device.id, data)
        ELSE
          data <- Buffer_Fetch(device.id)
          Write_Device(device, data)
        END IF
      END IF
    END FOR

    IF Interrupt_Pending() == TRUE THEN // service DMA completions promptly
      Service_Pending_Interrupts()
    END IF
    SLEEP(POLL_INTERVAL) // avoid busy-waiting, save CPU cycles
  END WHILE
END PROCEDURE

// Triggered asynchronously by hardware, independent of the polling loop
ISR DMA_Interrupt_Handler(device_id)
  status <- Check_DMA_Status(device_id)
  IF status == COMPLETE THEN
    Notify_Application(device_id, SUCCESS)
  ELSE
    Notify_Application(device_id, ERROR)
    Retry_Or_Log(device_id)
  END IF
  Free_DMA_Channel(device_id)
END ISR

// Reclassify a device at runtime if its throughput changes
PROCEDURE Monitor_And_Reclassify()
  WHILE System.running DO
    FOR EACH device IN device_list DO
      measured_rate <- Measure_Throughput(device)
    END FOR
  END WHILE
END PROCEDURE

```

```
    IF measured_rate != device.data_rate THEN
        device.data_rate <- measured_rate
        Classify_Device(device)
    END IF
END FOR
SLEEP(RECLASSIFY_INTERVAL)
END WHILE
END PROCEDURE
END MODULE
```